**ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH**
**TRƯỜNG ĐẠI HỌC BÁCH KHOA**
**KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH**

# BÁO CÁO THỰC HÀNH

## MÔN HỌC: INTERNET OF THINGS – CHƯƠNG TRÌNH THẠC SĨ

| Thông tin học viên | |
|---|---|
| Họ và tên | Đàm Quang Phục |
| Mã số học viên | 2570483 |
| Lớp / Ngành | CH01 |
| Email | phuc.damquang@hcmut.edu.vn |
| Học kỳ – Năm học | HK261 |
| Ngày nộp | 10/11/2026 |
| Kho mã nguồn (GitHub, nếu có) | https://github.com/dqpinfo/BK_IoT_Exercise.git |

## I. TỔNG HỢP ĐIỂM

Mỗi bài thực hành được đánh giá theo ba tiêu chí: mã nguồn (0.8 điểm), minh chứng kết quả (0.8 điểm) và giải thích, trả lời câu hỏi (0.4 điểm). Phần **Điểm đạt** và **Nhận xét** do giảng viên điền.

| Bài | Nội dung thực hành | Điểm tối đa | Điểm đạt | Nhận xét |
|---|---|---|---|---|
| 1 | Tạo tài khoản, thiết bị (device) và Dashboard trên CoreIOT | 2.0 | | |
| 2 | LED Blinky trên YoloUNO | 2.0 | | |
| 3 | Kết nối I2C với cảm biến DHT20 | 2.0 | | |
| 4 | Lập trình Semaphore trên YoloUNO | 2.0 | | |
| 5 | Thiết kế Webserver trên YoloUNO | 2.0 | | |
| | **TỔNG ĐIỂM** | **10.0** | | |

# II. ĐÁNH GIÁ CỦA GIẢNG VIÊN

| Nhận xét chung |
|---|
| |
| |
| |

| Điểm (bằng số) | Điểm (bằng chữ) | Kết quả |
|---|---|---|
| | | ☐ Đạt ☐ Không đạt |

*TP. Hồ Chí Minh, ngày [...] tháng [...] năm [20...]*

**Giảng viên chấm**

*(Ký và ghi rõ họ tên)*

# III. HƯỚNG DẪN THỰC HIỆN BÁO CÁO

- Học viên điền vào các vị trí được đánh dấu **[…]** và các khung để trống; xoá các khung "Hướng dẫn" trước khi nộp.
- **Mã nguồn:** chỉ dán phần mã được yêu cầu ở mỗi bài (không dán toàn bộ thư viện), giữ nguyên định dạng, có chú thích ở các dòng quan trọng. Mã nguồn đầy đủ có thể đưa lên GitHub và ghi đường dẫn ở trang đầu.
- **Hình ảnh minh chứng:** là ảnh chụp thực tế hoặc ảnh chụp màn hình của chính học viên; hình phải rõ nét, đọc được nội dung. Trên ảnh chụp màn hình nên thấy được tên tài khoản hoặc thông tin nhận diện học viên. **Không để lộ mật khẩu, access token, mật khẩu Wi-Fi.**
- Với các bài yêu cầu video, học viên ghi đường dẫn video (Google Drive, YouTube chế độ không công khai…) và đảm bảo giảng viên có quyền truy cập.
- Nộp báo cáo dạng PDF, đặt tên: [MSHV]_[HoTen]_BaoCaoThucHanh.pdf.

## Cam kết của học viên

Tôi xin cam kết các nội dung, mã nguồn và hình ảnh minh chứng trong báo cáo này do tôi tự thực hiện. Các nội dung tham khảo (tài liệu, mã nguồn mẫu, công cụ AI) đều đã được ghi rõ nguồn.

**Học viên**

*(Ký và ghi rõ họ tên)*

# IV. NỘI DUNG CÁC BÀI THỰC HÀNH

## BÀI 1. TẠO TÀI KHOẢN, THIẾT BỊ (DEVICE) VÀ DASHBOARD TRÊN COREIOT

**Mục tiêu:** Làm quen với nền tảng CoreIOT: đăng ký tài khoản, tạo thiết bị, gửi dữ liệu telemetry lên server và hiển thị trên dashboard.

**Yêu cầu thực hiện:**

- Tạo tài khoản trên CoreIOT và đăng nhập thành công.
- Tạo ít nhất 01 thiết bị (device), lấy access token của thiết bị.
- Gửi dữ liệu telemetry thử nghiệm (ví dụ nhiệt độ, độ ẩm) lên thiết bị vừa tạo.
- Thiết kế 01 dashboard có tối thiểu 02 widget hiển thị dữ liệu (biểu đồ, đồng hồ, thẻ giá trị…).

### *1.1. Mã nguồn*

Gửi telemetry bằng Python

```
// Topic: PUBLISH_TELEMETRY = "v1/devices/me/telemetry"
// Payload: {'temperature': temperature, 'humidity': humidity,
                        'longitude': POSITION_LONGITUDE, 'latitude':
POSITION_LATITUDE}
```

Mã nguồn

```
    client = mqttclient.Client(CLIENT_NAME)
    client.username_pw_set(username = ACCESS_TOKEN, password = None)

    client.on_connect = on_connect
    client.connect(BROKER_ADDRESS, BROKER_PORT)
    client.loop_start()

    client.on_subscribe = on_subscribe
    client.on_message = on_message

    temperature = 25
    humidity = 50

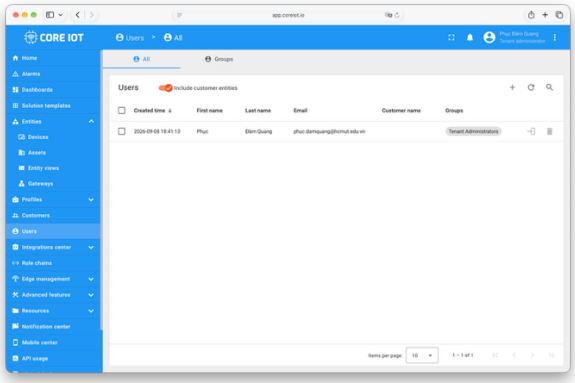
    while True:

        temperature = round(temperature + random.choice([-0.1, 0.1]), 1)
        humidity    = round(humidity + random.choice([-0.1, 0.1]), 1)
        print(f'temperature: {temperature} - humidity: {humidity}')

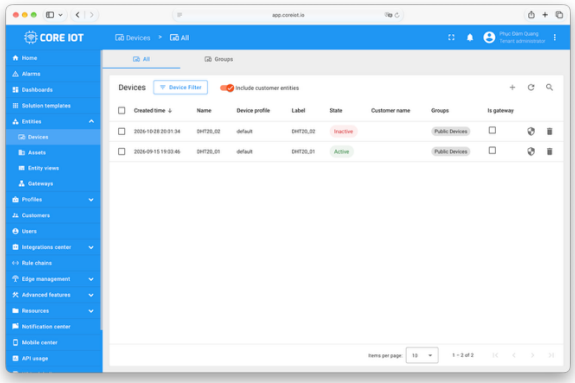
        collect_data = {'temperature': temperature, 'humidity': humidity,
                        'longitude': POSITION_LONGITUDE, 'latitude':
POSITION_LATITUDE}
        client.publish(PUBLISH_TELEMETRY, json.dumps(collect_data), 1)

        time.sleep(2)
```

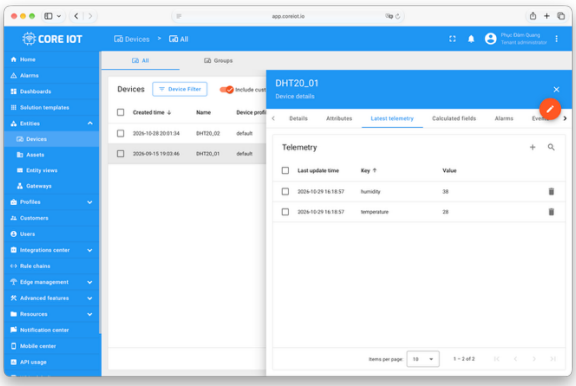
## 1.2. Hình ảnh minh chứng kết quả



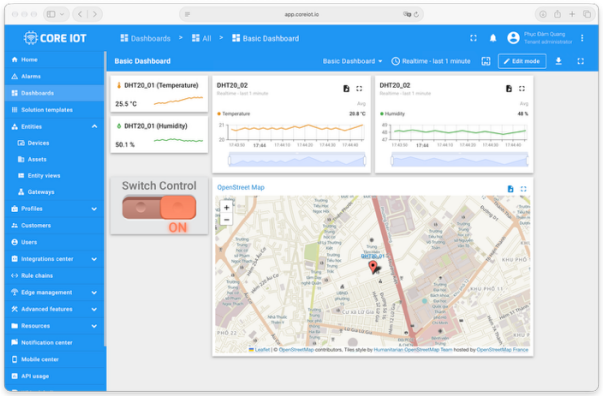
*Hình 1.1. Trang quản lý tài khoản sau khi đăng nhập*



*Hình 1.2. Thông tin các thiết bị đã tạo*



*Hình 1.3. Tab Latest telemetry hiển thị dữ liệu nhận được*



*Hình 1.4. Dashboard hoàn chỉnh với các widget*

## 1.3. Giải thích và trả lời câu hỏi

**Câu hỏi:** Nêu sự khác nhau giữa telemetry và attribute trên CoreIOT. Dữ liệu nhóm gửi lên thuộc loại nào?

```
telemetry: nơi publish dữ liệu liên tục theo thời gian thực.

attribute: nơi gửi thông tin, trạng thái, cấu hình của thiết bị. Chỉ gửi khi cần.

Dữ liệu nhóm bao gồm: temperature, humidity, longitude, latitude được gửi lên telemetry liên tục sau mỗi 2 giây.
```

## 1.4. Đánh giá của giảng viên

| STT | Tiêu chí đánh giá | Điểm tối đa | Điểm đạt |
|---|---|---|---|
| 1 | Mã nguồn đúng yêu cầu, rõ ràng, có chú thích | 0.8 | |
| 2 | Minh chứng kết quả đầy đủ, rõ ràng | 0.8 | |
| 3 | Giải thích, trả lời câu hỏi | 0.4 | |
| | **Tổng điểm bài** | **2.0** | |

| Nhận xét của giảng viên |
| --- |
| |
| |

# BÀI 2. LED BLINKY TRÊN YOLOUNO

**Mục tiêu:** Làm quen với môi trường lập trình cho mạch YoloUNO (ESP32-S3): biên dịch, nạp chương trình và điều khiển ngõ ra GPIO.

**Yêu cầu thực hiện:**

- Cài đặt môi trường lập trình (Arduino IDE hoặc PlatformIO) cho YoloUNO.
- Viết chương trình nhấp nháy LED với chu kỳ do học viên tự chọn (ví dụ sáng 1 giây, tắt 1 giây).
- In trạng thái LED ra Serial Monitor mỗi lần thay đổi.

## *2.1. Mã nguồn*

Chân LED và chu kỳ (ms)

```
#define INTERVAL  2000
#define LED_PIN   48
#define NEO_PIN   45
#define NEO_COUNT 1
```

Hàm setup()

```
void setup()
{
  // put your setup code here, to run once:
  Serial.begin(115200);

  xTaskCreate(led_blinky, "LED Control", 2048, NULL, 31, NULL);  // Task control
single LED
  xTaskCreate(neo_blinky, "NEO Control", 2048, NULL, 1, NULL);  // Task control LED
NEO
}
```

Hàm loop()

```
void loop()
{
  // put your main code here, to run repeatedly:
}
```

Task điều khiển single LED

```
void led_blinky(void *pvParameters)
{
  pinMode(LED_PIN, OUTPUT); // Initialize LED pin

  while(1)
  {
    digitalWrite(LED_PIN, HIGH);  // Turn ON LED
    Serial.println("Single LED => ON");
    vTaskDelay(INTERVAL);

    digitalWrite(LED_PIN, LOW);   // Turn OFF LED
    Serial.println("Single LED => OFF");
    vTaskDelay(INTERVAL);
  }
}
```

Task điều khiển NEO LED

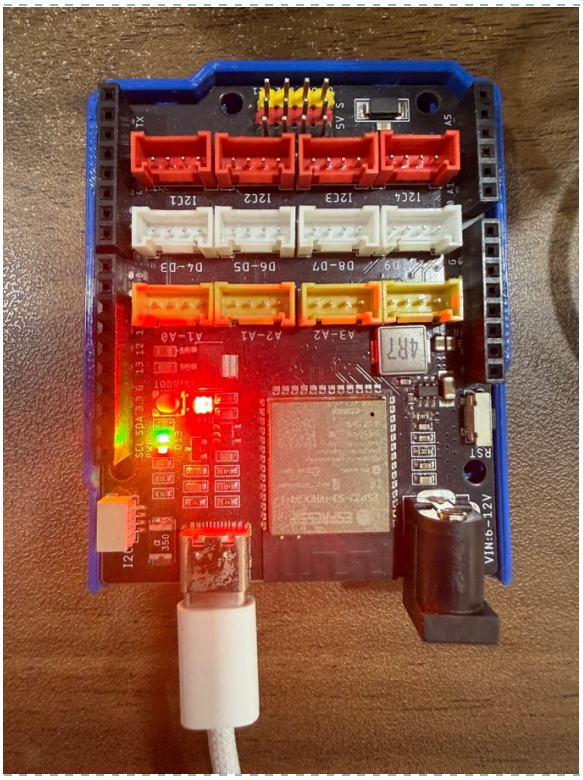
```
void neo_blinky(void *pvParameters)
{
    Adafruit_NeoPixel strip(NEO_COUNT, NEO_PIN, NEO_GRB + NEO_KHZ800);

    strip.begin();
    strip.clear();  // Set all pixels to off to start
    strip.show();

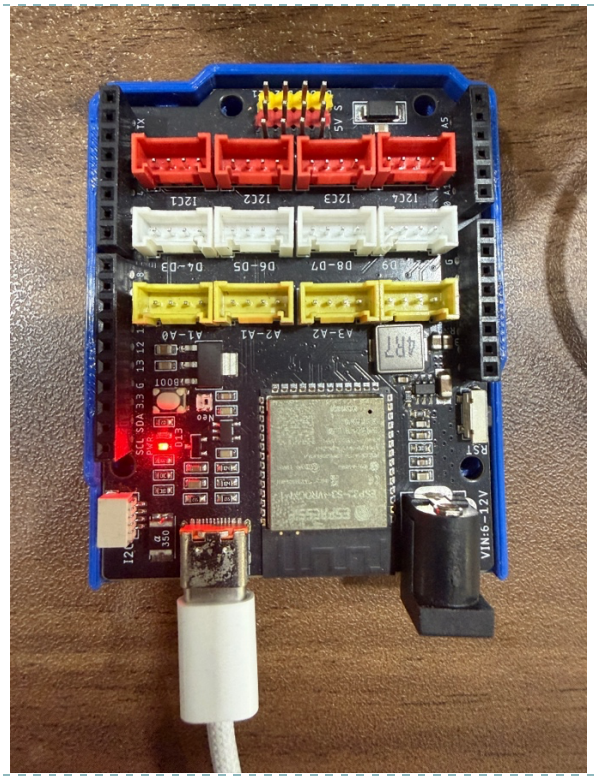
    while(1)
    {
        strip.setPixelColor(0, strip.Color(255, 0, 0)); // Set pixel 0 to red
        strip.show();                                   // Update the strip
        Serial.println("NEO LED => ON");
        vTaskDelay(INTERVAL);

        strip.setPixelColor(0, strip.Color(0, 0, 0));   // Set the pixel to off
        strip.show();                                   // Update the strip
        Serial.println("NEO LED => OFF");
        vTaskDelay(INTERVAL);
    }
}
```

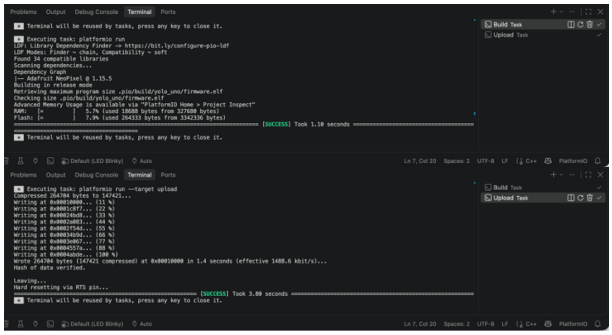
## 2.2. Hình ảnh minh chứng kết quả



*Hình 2.1. Mạch YoloUNO khi LED sáng*



*Hình 2.2. Mạch YoloUNO khi LED tắt*



*Hình 2.3. Kết quả biên dịch và nạp chương trình thành công*



*Hình 2.4. Log trạng thái LED trên Serial Monitor*

**Đường dẫn video minh chứng:**

https://github.com/dqpinfo/BK_IoT_Exercise/blob/af2b681191892db9c96793f48ba3a49ad558ff17/Report/LED_Blinky_720.mov

## *2.3. Giải thích và trả lời câu hỏi*

**Câu hỏi:** Hàm delay() ảnh hưởng thế nào đến các tác vụ khác của chương trình? Đề xuất cách nhấp nháy LED không dùng delay().

```
Hàm delay() sẽ block tất cả các tác vụ cho tới khi hết thời gian.

Cách nhấp nháy LED không dùng delay() là dùng hàm millis(), biến lastUpdated và interval. Mỗi lần lặp, kiểm tra nếu thời gian đã qua lớn hơn interval thì thực hiện thay đổi trạng thái LED; nếu không thì thoát. Như vậy chương trình sẽ không bị block.
```

## *2.4. Đánh giá của giảng viên*

| STT | Tiêu chí đánh giá | Điểm tối đa | Điểm đạt |
|---|---|---|---|
| 1 | Mã nguồn đúng yêu cầu, rõ ràng, có chú thích | 0.8 | |
| 2 | Minh chứng kết quả đầy đủ, rõ ràng | 0.8 | |
| 3 | Giải thích, trả lời câu hỏi | 0.4 | |
| | **Tổng điểm bài** | **2.0** | |

| Nhận xét của giảng viên |
|---|
| |
| |

# BÀI 3. KẾT NỐI I2C VỚI CẢM BIẾN DHT20

**Mục tiêu:** Hiểu và sử dụng giao tiếp I2C để đọc dữ liệu nhiệt độ, độ ẩm từ cảm biến DHT20.

**Yêu cầu thực hiện:**

- Kết nối cảm biến DHT20 với YoloUNO qua bus I2C (SDA, SCL, VCC, GND).
- Quét bus I2C và xác định địa chỉ của cảm biến (DHT20 có địa chỉ mặc định 0x38).
- Đọc nhiệt độ, độ ẩm định kỳ và in ra Serial Monitor.
- (Khuyến khích) Gửi dữ liệu đọc được lên dashboard CoreIOT đã tạo ở Bài 1.

## *3.1. Mã nguồn*

Chân pin:

```
#define SDA_PIN 11
#define SCL_PIN 12
```

Địa chỉ I2C: 0x38

Khởi tạo bus I2C và các task

```
void setup()
{
  // put your setup code here, to run once:
  Serial.begin(115200);

  Wire.begin(SDA_PIN, SCL_PIN);
  dht.begin();

  delay(1000);  // Wait for sensor stable

  //xTaskCreate(task_scan_I2C_bus, "Scan I2C Bus", 2048, NULL, 31, NULL); // Task
scan I2C bus
  xTaskCreate(task_read_DHT20_info, "Read DHT20 Info", 4096, NULL, 10, NULL); //
Task read DHT20 information
}
```

Task quét I2C bus

```
void task_scan_I2C_bus(void *pvParameters)
{
  byte address, error;

  while (1)
  {
    for (address = 1; address < 127; address++)
    {
      Wire.beginTransmission(address);
      error = Wire.endTransmission();

      if (error == 0)
      {
        Serial.print("\nI2C device found at: 0x");
        if (address < 16) Serial.print("0");
        Serial.println(address, HEX);
      }
      else
```

```
      {
        Serial.print(".");
      }
    }

    // Scan every 5 seconds
    vTaskDelay(5000);
  }
}
```

Task đọc thông tin từ DHT20

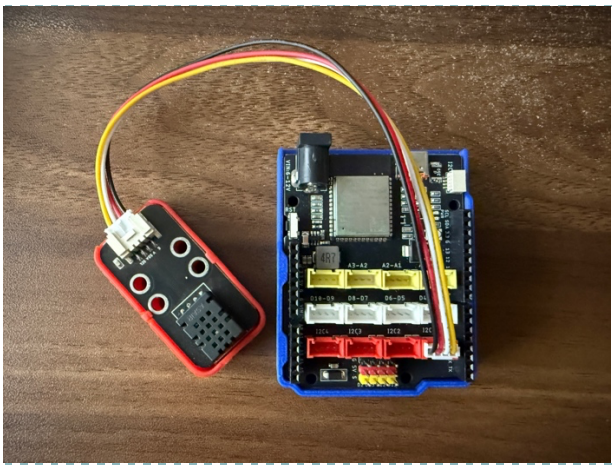
```
void task_read_DHT20_info(void *pvParameters)
{
  while (1)
  {
    dht.read();

    temperature = dht.getTemperature();
    humidity = dht.getHumidity();

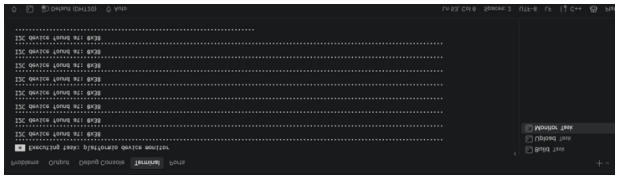
    if (isnan(temperature) || isnan(humidity))
    {
      Serial.println("Read DHT20 sensor failed");
    }
    else
    {
      sprintf(buffer, "Temperature: %.1f °C - Humidity: %.1f %%", temperature,
humidity);
      Serial.println(buffer);
    }

    vTaskDelay(3000);
  }
}
```

## *3.2. Hình ảnh minh chứng kết quả*



*Hình 3.1. Sơ đồ / hình ảnh kết nối DHT20 với YoloUNO*



*Hình 3.2. Kết quả quét địa chỉ I2C*




*Hình 3.3. Giá trị nhiệt độ, độ ẩm trên Serial Monitor*

*[Chèn hình tại đây]*

*Hình 3.4. (Khuyến khích) Dữ liệu DHT20 trên dashboard CoreIOT*

## *3.3. Giải thích và trả lời câu hỏi*

**Câu hỏi:** Giải thích vai trò của điện trở kéo lên (pull-up) trên bus I2C. Điều gì xảy ra nếu hai thiết bị trên bus có cùng địa chỉ?

```
Điện trở kéo lên (pull-up) giúp tạo trạng thái mặc định ở mức cao (HIGH) ổn định. Nếu không có điện trở kéo lên này, đường tín hiệu sẽ bị thả nổi khiến thiết bị đọc dữ liệu không chính xác.

Trên bus có 2 thiết bị có cùng địa chỉ, khi master muốn giao tiếp với địa chỉ này thì sẽ có 2 thiết bị cùng phản hồi gây xung đột tín hiệu.
```

## *3.4. Đánh giá của giảng viên*

| STT | Tiêu chí đánh giá | Điểm tối đa | Điểm đạt |
|---|---|---|---|
| 1 | Mã nguồn đúng yêu cầu, rõ ràng, có chú thích | 0.8 | |
| 2 | Minh chứng kết quả đầy đủ, rõ ràng | 0.8 | |
| 3 | Giải thích, trả lời câu hỏi | 0.4 | |
| | **Tổng điểm bài** | **2.0** | |

| Nhận xét của giảng viên |
|---|
| |
| |

# BÀI 4. LẬP TRÌNH SEMAPHORE TRÊN YOLOUNO

**Mục tiêu:** Sử dụng semaphore của FreeRTOS để đồng bộ hóa và bảo vệ tài nguyên dùng chung giữa các tác vụ.

**Yêu cầu thực hiện:**

- Tạo tối thiểu 02 task FreeRTOS chạy đồng thời.
- Sử dụng semaphore (binary, counting hoặc mutex) để đồng bộ các task, ví dụ: task đọc cảm biến báo cho task hiển thị/gửi dữ liệu, hoặc bảo vệ tài nguyên dùng chung (Serial, biến toàn cục).
- In log thể hiện rõ thứ tự take/give semaphore giữa các task.

## *4.1. Mã nguồn*

```
Loại semaphore sử dụng: Binary Semaphore

Các task: [Tên task – độ ưu tiên – chức năng]
   1. Monitor Button, 31, Theo dõi trạng thái nút nhấn, nếu nhấn sẽ Give.
   2. Blink LED, 1, Chờ cho tới khi Take được thì thực hiện blink LED 3 lần.
```

setup()

```
void setup()
{
  // put your setup code here, to run once:
  Serial.begin(115200);
  delay(1000);

  pinMode(BUTTON_PIN, INPUT_PULLUP);
  pinMode(LED_PIN, OUTPUT);

  // Created empty (0), task Take has to wait
  xBinarySemaphore = xSemaphoreCreateBinary();

  if (xBinarySemaphore == NULL)
  {
    Serial.println("Create semaphore failed (out of heap)");
    return;
  }

  // Stack size is in BYTES on the ESP32 port
  xTaskCreate(task_monitor_button, "Monitor Button", 4096, NULL, 31, NULL);
  xTaskCreate(task_blink_led, "Blink LED", 4096, NULL, 1, NULL);
}
```

loop()

```
void loop()
{
  // put your main code here, to run repeatedly:
}
```

Task monitor button

```
void task_monitor_button(void *pvParameters)
{
  while (1)
```

```
  {
    // Button pressed (active-low)
    if (gpio_get_level(BUTTON_PIN) == 0)
    {
      vTaskDelay(pdMS_TO_TICKS(50)); // Simple debounce

      if (gpio_get_level(BUTTON_PIN) == 0)
      {
        // Give the semaphore to blink LED task
        if (xSemaphoreGive(xBinarySemaphore) == pdTRUE)
          Serial.println("Give -> OK");
        else
          Serial.println("Give ignored (semaphore full)");

        // Wait for release: 1 press = 1 Give
        while (gpio_get_level(BUTTON_PIN) == 0)
          vTaskDelay(pdMS_TO_TICKS(10));
      }
    }

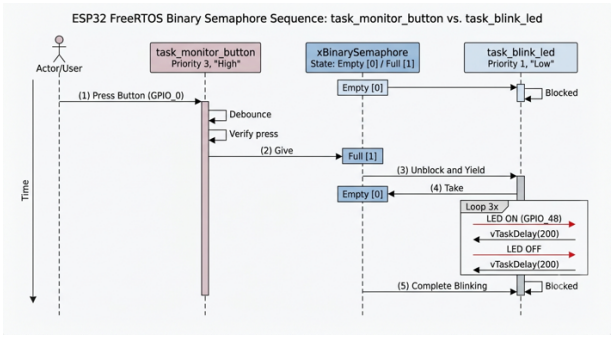
    // Polling every 10 ms
    vTaskDelay(pdMS_TO_TICKS(10));
  }
}
```

Task blink LED 3 lần

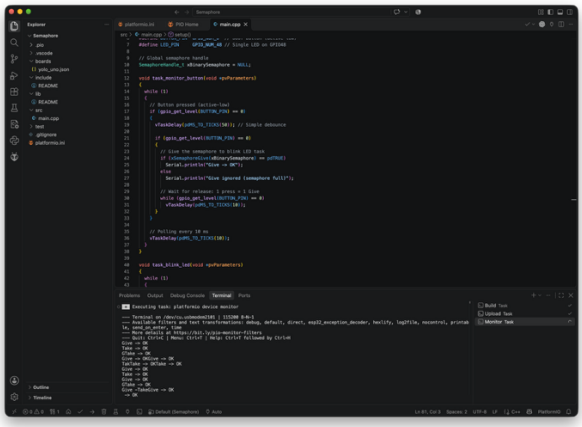
```
void task_blink_led(void *pvParameters)
{
  while (1)
  {
    // Blocked here (0% CPU) until monitor button task gives
    if (xSemaphoreTake(xBinarySemaphore, portMAX_DELAY) == pdTRUE)
    {
      Serial.println("Take -> OK");

      // Blink LED 3 times
      for (int i = 0; i < 3; i++)
      {
        gpio_set_level(LED_PIN, 1);
        vTaskDelay(pdMS_TO_TICKS(200));
        gpio_set_level(LED_PIN, 0);
        vTaskDelay(pdMS_TO_TICKS(200));
      }
    }
  }
}
```

## 4.2. Hình ảnh minh chứng kết quả




*Hình 4.1. Sơ đồ tương tác giữa các task và semaphore*



*Hình 4.2. Log Serial Monitor thể hiện quá trình đồng bộ*

## 4.3. Giải thích và trả lời câu hỏi

**Câu hỏi:** Vì sao chọn loại semaphore này? So sánh binary semaphore và mutex, nêu hiện tượng đảo ưu tiên (priority inversion) và cách mutex hạn chế hiện tượng đó.

Em chọn Binary Semaphore trong bài tập này vì nút nhấn có 2 trạng thái nhấn / nhả và không quan tâm nút được nhấn trong quá trình LED nháy 3.
Khi nút được nhấn thì nháy LED 3 lần rồi quay lại chờ cho tới khi được Take.

So sánh binary semaphore và mutex
- Về mục đích sử dụng: semaphore binary dùng để đồng bộ các task khác nhau trong khi mutex thường được dùng để bảo vệ tài nguyên dùng chung.
- Về Give và Take: semaphore binary có thể được give ở task này và tack ở task khác, trong khi mutex thì Give và Take được thực hiện trong cùng 1 task (làm xong 1 tác vụ nào đó).
- Về cơ chế chống đảo ưu tiên: semaphore binary không hỗ trợ, mutex có hỗ trợ.

Hiện tượng đảo ưu tiên (priority inversion): hiện tượng này xảy ra khi 1 task có độ ưu tiên cao hơn bị kẹt lại sau task có độ ưu tiên thấp hơn.

Mutex giải quyết hiện tượng đảo ngược ưu tiên bằng cách kế thừa độ ưu tiên (priority inheritance). Khi task có độ ưu tiên thấp đang lock tài nguyên mà task có độ ưu tiên cao hơn đang chờ thì task có độ ưu tiên thấp sẽ kế thừa độ ưu tiên của task có độ ưu tiên cao hơn.

## 4.4. Đánh giá của giảng viên

| STT | Tiêu chí đánh giá | Điểm tối đa | Điểm đạt |
|---|---|---|---|
| 1 | Mã nguồn đúng yêu cầu, rõ ràng, có chú thích | 0.8 | |
| 2 | Minh chứng kết quả đầy đủ, rõ ràng | 0.8 | |
| 3 | Giải thích, trả lời câu hỏi | 0.4 | |
| | **Tổng điểm bài** | **2.0** | |

| Nhận xét của giảng viên |
| --- |
| |
| |

# BÀI 5. THIẾT KẾ WEBSERVER TRÊN YOLOUNO

**Mục tiêu:** Hiện thực webserver chạy trực tiếp trên YoloUNO để giám sát và điều khiển thiết bị trong mạng cục bộ.

**Yêu cầu thực hiện:**

- Kết nối YoloUNO vào mạng Wi-Fi (chế độ Station) hoặc phát Wi-Fi (chế độ Access Point).
- Thiết kế trang web hiển thị dữ liệu cảm biến (ví dụ DHT20 ở Bài 3).
- Trang web có chức năng điều khiển (ví dụ bật/tắt LED ở Bài 2).
- Dữ liệu trên trang web được cập nhật định kỳ hoặc theo thời gian thực.

## *5.1. Mã nguồn*

```
Chế độ Wi-Fi: AP mode   Địa chỉ IP: 192.168.4.1
Chế độ Wi-Fi: STA mode  Địa chỉ IP: 192.168.31.101
```

setup()

```
void setup()
{
  // put your setup code here, to run once:
  Serial.begin(115200);

  Wire.begin(SDA_PIN, SCL_PIN);
  dht.begin();

  delay(1000);  // Wait for sensor stable

  // Tạo semaphore trước khi task dùng
  xBinarySemaphoreInternet = xSemaphoreCreateBinary();

  // Khởi động các task
  xTaskCreate(task_wifi, "task WiFi", 8192, NULL, 31, NULL);
  xTaskCreate(task_dht20, "task DHT20", 4096, NULL, 30, NULL);
  xTaskCreate(task_LED, "task LED", 4096, NULL, 29, NULL);
}
```

loop()

```
void loop()
{
  // put your main code here, to run repeatedly:
}
```

Task điều khiển LED

```
void task_LED(void *pvParameters)
{
  pinMode(LED_PIN, OUTPUT);

  Adafruit_NeoPixel strip(NEO_COUNT, NEO_PIN, NEO_GRB + NEO_KHZ800);
  strip.begin();
  strip.clear();  // Set all pixels to off to start
  strip.show();

  while (1)
```

```
  {
    if (glob_singleLEDState)
    {
      digitalWrite(LED_PIN, HIGH);  // Turn ON LED
    }
    else
    {
      digitalWrite(LED_PIN, LOW);   // Turn OFF LED
    }

    if (glob_neoLEDState)
    {
      strip.setPixelColor(0, strip.Color(255, 0, 0)); // Set pixel 0 to red
      strip.show();                                   // Update the strip
    }
    else
    {
      strip.setPixelColor(0, strip.Color(0, 0, 0));   // Set the pixel to off
      strip.show();                                   // Update the strip
    }

    vTaskDelay(100);
  }
}
```

Task đọc thông tin từ DHT20

```
void task_dht20(void *pvParameters)
{
  while (1)
  {
    dht.read();

    glob_temperature = dht.getTemperature();
    glob_humidity = dht.getHumidity();

    if (isnan(glob_temperature) || isnan(glob_humidity))
    {
      Serial.println("Read DHT20 sensor failed");
    }
    else
    {
      Serial.printf("Temperature: %.1f °C - Humidity: %.1f %%\n", glob_temperature,
glob_humidity);
    }

    vTaskDelay(3000);
  }
}
```

Task điều khiển Wi-Fi

```
void task_wifi(void *pvParameters)
{
    pinMode(BOOT_BUTTON_PIN, INPUT_PULLUP);
```

```
    while (1)
    {
        handleClient();

        // Switch to AP Mode when boot button pressed
        if (digitalRead(BOOT_BUTTON_PIN) == LOW)
        {
            vTaskDelay(300); // Simple debounce

            if (digitalRead(BOOT_BUTTON_PIN) == LOW)
            {
                if (isSTDMode)
                {
                    startAP();
                    setupServer();
                }
            }
        }

        // STA Mode
        if (isSTDMode)
        {
            if (WiFi.status() == WL_CONNECTED)
            {
                if (!isWifiConnected)
                {
                    Serial.print("STA IP address: ");
                    Serial.println(WiFi.localIP());

                    setupServer();
                    isWifiConnected = true;
                }

                xSemaphoreGive(xBinarySemaphoreInternet);
            }
            else if (millis() - lastConnectWifiTime > 5000)
            {
                // Reconnect to wifi every 5 seconds
                connectToWiFi();
            }
        }

        vTaskDelay(20); // Avoid watchdog reset
    }
}
```

Khởi tạo Wi-Fi: AP mode

```
void startAP()
{
    isSTDMode = false;
    WiFi.mode(WIFI_AP);
    WiFi.softAP(ssid.c_str(), password.c_str());
```

```
    Serial.printf("AP IP address:  %s\n", WiFi.softAPIP().toString());
}
```

Khởi tạo Wi-Fi: STA mode

```
void connectToWiFi()
{
    Serial.printf("Connecting to %s\n", wifi_ssid.c_str());

    isWifiConnected = false;
    lastConnectWifiTime = millis();

    isSTDMode = true;
    WiFi.mode(WIFI_STA);

    if (wifi_password.isEmpty())
    {
        WiFi.begin(wifi_ssid.c_str());
    }
    else
    {
        WiFi.begin(wifi_ssid.c_str(), wifi_password.c_str());
    }
}
```

Khởi tạo WebServer

```
void setupServer()
{
  server.on("/", HTTP_GET, handleRoot);
  server.on("/toggle", HTTP_GET, handleToggle);
  server.on("/sensors", HTTP_GET, handleSensors);
  server.on("/settings", HTTP_GET, handleSettings);
  server.on("/connect", HTTP_GET, handleConnect);
  server.begin();
}
```

WebServer: handle DHT20 sensor

```
void handleSensors()
{
  String json = "{\"temp\":" + String(glob_temperature) + ",\"hum\":" +
String(glob_humidity) + "}";
  server.send(200, "application/json", json);
}
```

WebServer: handle LED switches

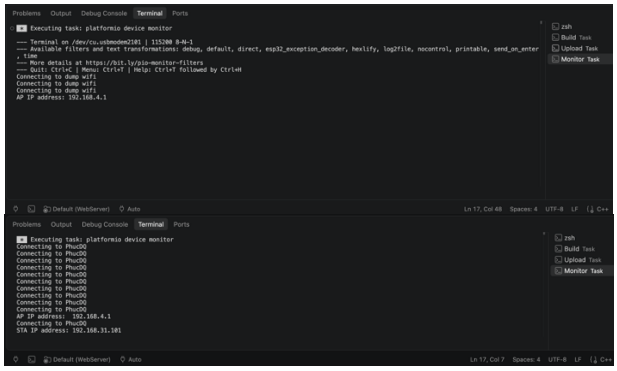
```
void handleToggle()
{
  int led = server.arg("led").toInt();

  if (led == 1)
  {
    glob_singleLEDState = !glob_singleLEDState;
  }
  else if (led == 2)
```

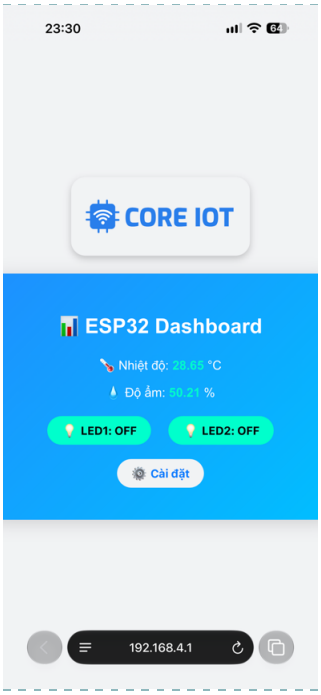
```
  {
    glob_neoLEDState = !glob_neoLEDState;
  }

  server.send(200, "application/json",
              "{\"led1\":\"" + String(glob_singleLEDState ? "ON" : "OFF") +
              "\",\"led2\":\"" + String(glob_neoLEDState ? "ON" : "OFF") + "\"}");
}
```

## 5.2. Hình ảnh minh chứng kết quả



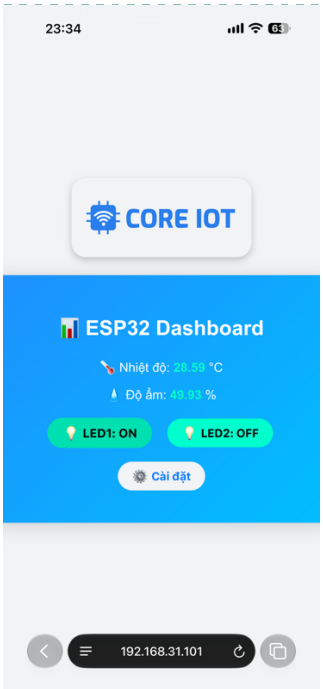
*Hình 5.1. Serial Monitor hiển thị địa chỉ IP của webserver (AP and STA mode)*




*Hình 5.2. Giao diện trang web AP mode*




*Hình 5.3. Giao diện trang web WiFi setting*




*Hình 5.4. Giao diện trang web STA mode*

**Đường dẫn video minh chứng:**

https://github.com/dqpinfo/BK_IoT_Exercise/blob/5b025cca5f4a29623374f741ea31125ad802b0b0/Report/WebServer_720.mov

### *5.3. Giải thích và trả lời câu hỏi*

**Câu hỏi:** Trình bày cơ chế cập nhật dữ liệu trên trang web (tải lại trang, AJAX/fetch định kỳ hay WebSocket). Nêu ưu, nhược điểm của cơ chế đã chọn.

*Cơ chế cập nhật dữ liệu được sử dụng là AJAX/Fetch định kỳ (Polling) kết hợp với cập nhật bất đồng bộ qua sự kiện người dùng (Event-driven Fetch).*

*Cập nhật dữ liệu cảm biến định kỳ: Đoạn mã JavaScript sử dụng hàm setInterval kết hợp với fetch để yêu cầu dữ liệu từ endpoint /sensors sau mỗi 3 giây.*

*Cập nhật trạng thái thiết bị (LED) khi bấm nút: Khi người dùng tương tác bấm nút bật/tắt LED, hàm toggleLED gọi fetch tới đường dẫn /toggle để thay đổi trạng thái và nhận về dữ liệu JSON để cập nhật giao diện ngay lập tức.*

*Ưu và nhược điểm của cơ chế đã chọn (AJAX/Fetch định kỳ)*
***Ưu điểm:***
** Dễ hiện thực và triển khai: Giao thức HTTP/HTTPS truyền thống rất dễ cấu hình trên các vi điều khiển tài nguyên hạn chế như ESP32 thông qua thư viện WebServer.h.*
** Không làm gián đoạn trải nghiệm người dùng (Smooth UI): Chỉ có dữ liệu cảm biến hoặc trạng thái nút nhấn thay đổi được tải ngầm và cập nhật vào DOM, trang web không bị nhấp nháy hay tải lại toàn bộ như cách dùng cơ chế tải lại trang truyền thống.*
** Tiết kiệm tài nguyên kết nối cho ESP32: Máy chủ web chỉ phản hồi khi nhận được yêu cầu (request) từ client, không phải duy trì kết nối mở liên tục cho từng trình duyệt như WebSocket.*
***Nhược điểm:***
** Có độ trễ (Latency): Vì dùng cơ chế hỏi vòng (polling) định kỳ (ví dụ mỗi 3 giây), nếu dữ liệu cảm biến thay đổi ngay sau khi vừa quét xong, người dùng sẽ phải chờ đến chu kỳ tiếp theo mới thấy được giá trị cập nhật mới.*
** Lãng phí băng thông/tài nguyên khi không cần thiết: Máy chủ vẫn phải liên tục xử lý các yêu cầu /sensors gửi đến theo chu kỳ đều đặn ngay cả khi giá trị nhiệt độ và độ ẩm hoàn toàn không có sự thay đổi nào so với lần đọc trước.*

### *5.4. Đánh giá của giảng viên*

| STT | Tiêu chí đánh giá | Điểm tối đa | Điểm đạt |
|---|---|---|---|
| 1 | Mã nguồn đúng yêu cầu, rõ ràng, có chú thích | 0.8 | |
| 2 | Minh chứng kết quả đầy đủ, rõ ràng | 0.8 | |
| 3 | Giải thích, trả lời câu hỏi | 0.4 | |
| | **Tổng điểm bài** | **2.0** | |

| Nhận xét của giảng viên |
|---|
| |
| |